

Relazione di Laboratorio: Random Walk

GABRIELE FRUGONI (713463) e MATTEO LEONARDI (713343)

gruppo B2-3 tavolo 9

22 aprile 2026

1 Scopo dell'esperienza

Simulare l'algoritmo di random walk in due dimensioni e studiarne le proprietà statistiche

2 Cenni teorici

Un algoritmo di random walk consiste in un percorso generato casualmente; nel nostro caso studieremo un random walk in due dimensioni con un passo che per comodità fisseremo ad 1 (in unità arbitraria). Partendo dall'origine del piano (0,0) l'algoritmo genererà un angolo $\phi \in [0, 2\pi]$. Dopodiché si muoverà di un passo verso la direzione ϕ , operativamente vuol dire che aggiornerà le sue coordinate (x,y) in modo che:

$$\begin{cases} x \rightarrow x + \cos(\phi) \\ y \rightarrow y + \sin(\phi) \end{cases} \quad (1)$$

Iterando questa operazione n volte si ottiene un percorso simile a Figura 1 dove la posizione finale è in coordinate (x_{finale}, y_{finale}) ovvero la somma degli spostamenti elementari per ogni asse

$$x_{finale} = \sum_{i=1}^n \cos(\phi_i), \quad y_{finale} = \sum_{i=1}^n \sin(\phi_i) \quad (2)$$

Per il Teorema del Limite Centrale: la somma di variabili casuali indipendenti, identicamente distribuite e con varianza finita¹ tende a distribuirsi secondo una legge

¹Siamo sicuri della varianza finita poichè l'angolo ϕ viene generato in un intervallo finito e il passo è unitario. Questo determina un dominio finito entro il quale giace il random pattern

normale, indipendentemente dalla distribuzione originale delle singole variabili. Ciò vuol dire che per n elementi le coordinate x e y sono distribuite come una Gaussiana con media $= 0$ e varianza $\sigma^2 = \frac{n}{2}$.

Definendo con $d := \sqrt{x_{finale}^2 + y_{finale}^2}$ la distanza dall'origine a cui si arriva dopo un ciclo di random walk, si può notare come d^2 sia distribuito come un χ^2 con due gradi di libertà, ovvero una distribuzione esponenziale di media n , poichè somma di due gaussiane al quadrato.

$$p(d^2; n) = \frac{1}{n} e^{-\frac{d^2}{n}} \quad (3)$$

Attraverso un cambio di variabile è possibile risalire alla distribuzione della d che coincide con la distribuzione di un χ riscalata (la varianza è n e non 1)

$$p(d; n) = \frac{2d}{n} e^{-\frac{d^2}{n}} \quad (4)$$

Calcolando il valor medio dell'Equazione 4 si ottiene:

$$E[d] = \int_0^{+\infty} d \cdot p(d; n) dd = \int_0^{+\infty} d \left(\frac{2d}{n} e^{-\frac{d^2}{n}} \right) dd$$

Che svolgendo i calcoli risulta:

$$E[d] = \frac{\sqrt{\pi}}{2} \sqrt{n} \quad (5)$$

3 Materiale a disposizione

Ambiente Python esteso dalle librerie:

- numpy: per funzioni matematiche complesse, per la gestione semplificata degli array e per la generazione di numeri casuali
- matplotlib: per la rappresentazione grafica dei grafici
- scipy.stats: per calcolare la p-value

4 Codice

Di seguito riportato il codice e i relativi commenti

```

1 import numpy as np
2 from matplotlib import pyplot as plt
3 from scipy.stats import chi2
4 import matplotlib as mpl
5 import sys
6
7 #sezione totalmente opzionale usata per stampare i grafici in
formato pgf
8 if len(sys.argv) > 1:
9     img = sys.argv[1]          #bool, se False usa plt.show, se True
usa stmpa pgf
10 else:
11     img = False                #se fallisce l'inserimento
12
13 #convertitore da stringa a bool
14 if img == 'True' or img == 'true':
15     img = True
16 elif img == 'False' or img == 'false':
17     img = False
18
19 if img:
20     mpl.use("pgf")
21
22     plt.rcParams.update({
23         "font.family": "serif",
24         "text.usetex": True,
25         "pgf.rcfonts": False,
26     })
27
28
29 #definizione funzione calcolo step finale random walk
30 def random_walk(num_steps=1000):
31     #genera randomicamente un angolo tra 0 e 2pi-greco
32     phi = np.random.uniform(0., 2. * np.pi, size=num_steps)
33     #per ogni angolo somma in due array distinti il coseno (array x
) ed il seno (array y).
34     # In questo modo ottengo direttamente le coordinate del punto
finale alleggerendo l'esecuzione
35     x = np.sum(np.cos(phi))
36     y = np.sum(np.sin(phi))
37     return x, y
38
39 #definizione funzioni per le distribuzioni
40 def gauss(x, A, mu, sigma):
41     return A / sigma / np.sqrt(2. * np.pi) * np.exp(-(x - mu) ** 2 /
42         (2. * sigma ** 2))

```

```

42
43 def distribuzione_esponenziale(d, larg_bin):
44     return (np.exp(-d/n) / n) * len(x) * larg_bin
45
46 def distribuzione_chi(d, larg_bin, n_passi, N_tot):
47     return ((2 * d / n_passi) * np.exp(-(d**2) / n_passi)) * N_tot
48     * larg_bin
49
50 #dichiarazione costanti
51 N = 100000 #numero di run di random walk
52 n = 1000 #nuper di passi in una singola run
53 bin_width = 2. #larghezza dei bin del istogramma
54 della distribuzione della x e del istogramma della distribuzione
55 della d
56 d2_bin_width = 10. #larghezza dei bin del istogramma
57 della distribuzione della d^2
58 range_x_dist = 80. #range dati osservati nel
59 istogramma della x e della d (essendo centrato in 0->[-
60 range_x_dist, +range_x_dist])
61 range_d2_dist = 4000. #range dati osservati nel
62 istogramma della d^2 (non essendo centrato in 0->[0,
63 range_d2_dist])
64
65 #-- costanti per lo studio di d(n) --
66 max_n = 1000 #
67 min_n = 10 #
68 step_width = 20 #
69 run_per_n = 200 #numero di run che esegue per ogni
70 step
71
72 #suddivide l'intervallo da min_n a max_n in steps di grandezza
73 step_width
74 #es: se min_n = 0, max_n = 50 e step_width = 10 --> val_n = ([0,
75 10, 20, 30, 40, 50])
76 val_n = np.arange(min_n, max_n, step_width)
77
78 #richiamo il random walk N volte
79 x = []
80 y = []
81 for i in range(N):
82     x_i, y_i = random_walk(n)
83     x.append(x_i)
84     y.append(y_i)
85
86 #####Studio della distribuzione della x#####
87 fig1 = plt.figure(1)
88

```

```

77 #crea l'istogramma per graficare le x osservate
78 x_osservate, x_bins, _ = plt.hist(x, bins=np.arange(-range_x_dist,
    range_x_dist, bin_width), label="osservati")
79
80 #array con la coordinata x dei centri di tutti i bin
81 x_bin_centers = (x_bins[:-1] + x_bins[1:]) / 2.
82
83 #trova i valori di aspettazione della x dalla formula di una
    distribuzione gaussiana
84 x_aspettate = gauss(x_bin_centers, N * bin_width, 0., np.sqrt(n /
    2.))
85
86 #test chi2
87 dof_1 = len(x_osservate) - 1 #n di bin - 1 (vincolo sul numero
    totale di N)
88 chi2_1 = np.sum((x_osservate - x_aspettate) ** 2. / x_aspettate)
89 p_value_1 = chi2.sf(chi2_1, dof_1)
90
91 #calcolo e deviazione standard per lo studio della bonta' della
    distribuzione
92 mediaX = np.mean(x)
93 devStd = np.std(x)
94
95 #plot grafico (i plot da riga 97 a 100 hanno solo scopo estetico
    per la leggenda)
96 plt.plot(x_bin_centers, x_aspettate, 'r-', label=rf'aspettati')
97 plt.plot([], [], ' ', label=rf'$\chi^2$ / dof: {chi2_1:.2f} / {
    dof_1}')
98 plt.plot([], [], ' ', label=rf'p-value: {p_value_1:.2f}')
99 plt.plot([], [], ' ', label=rf'Media x: {mediaX:.2f}')
100 plt.plot([], [], ' ', label=rf'Dev std: {devStd:.2f} ' '\n' rf'
    aspettata: {np.sqrt(n / 2):.2f}')
101 plt.legend()
102 plt.xlabel('Coord x')
103 plt.ylabel('Frequenza')
104
105 #####studio del d^2#####
106 fig2, (ax1, ax2) = plt.subplots(2, 1, sharex=True, height_ratios
    =[3, 1])
107
108 #calcolo array delle distanze al quadrato [d^2 = x^2 + y^2]
109 d2 = np.empty(len(x))
110 for i in range(len(x)):
111     d2[i] = x[i]**2 + y[i]**2
112
113 #crea l'istogramma per la distribuzione delle distanze al quadrato

```

```

114 d2_oss, d2_x_bins, _ = ax1.hist(d2, bins=np.arange(0.,
    range_d2_dist, d2_bin_width), label=rf'$d^2$ osservata')
115
116 #coordinate x sei centri di ogni bin dell'istogramma
117 d2_x_bin_centers = (d2_x_bins[:-1] + d2_x_bins[1:]) / 2.
118
119 #calcola i valori di aspettazione dal modello esponenziale
120 d2_aspettati = distribuzione_esponenziale(d2_x_bin_centers,
    d2_bin_width)
121
122 #calcolo dei residui
123 d2_residui = d2_oss - d2_aspettati
124
125 #sto studiando una poissioniana -> varianza = media -> errore
    statistico sul bin e' la deviazione standard -> errore stimato =
    radice(n)
126 d2_sigma_residui = np.sqrt(d2_oss)
127
128 #test del chi2
129 dof_2 = len(d2_oss) - 1 #n di bin - 1 (vincolo sul numero totale di
    N)
130 chi2_2 = np.sum(d2_residui **2 / d2_aspettati)
131 p_value_2 = chi2.sf(chi2_2, dof_2)
132
133 #plot del secondo grafico + residui
134 ax1.plot(d2_x_bin_centers, d2_aspettati, "r-", color="red", label=rf
    '$d^2$ aspettati (distribuzione esponenziale)')
135 ax1.plot([], [], ' ', label=rf'$\chi^2$ / dof: {chi2_2:.2f} / {
    dof_2}')
136 ax1.plot([], [], ' ', label=rf'p-value: {p_value_2:.2f}')
137 ax1.legend()
138 ax1.set_ylabel('Frequenza')
139 ax2.axhline(0, color='black', linestyle='dashed')
140 ax2.errorbar(d2_x_bin_centers, d2_residui, yerr=d2_sigma_residui,
    fmt='.')
141 ax2.grid()
142 ax2.set_xlabel(rf'$d^2$')
143 ax2.set_ylabel('Residui (osservati - attesi)')
144
145 #####studio della d#####
146 fig3, (ax3, ax4) = plt.subplots(2, 1, sharex=True, height_ratios
    =[3, 1])
147
148 #array delle distanze
149 d = np.sqrt(d2)
150

```

```

151 #crea l'istogramma della distribuzione delle distanze
152 d_oss, d_x_bins, _ = ax3.hist(d, bins=np.arange(0., 80., bin_width)
    , label=rf'$d$ osservati')
153
154 #array delle coordinate x dei centri dei bin
155 d_x_bin_centers = (d_x_bins[:-1] + d_x_bins[1:]) / 2.
156
157 #calcola i valori di aspettazione dalla distribuzione del chi2
158 d_aspettati = distribuzione_chi(d_x_bin_centers, bin_width, n, len(
    x))
159
160 #calcolo dei residui
161 d_residui = d_oss - d_aspettati
162
163 #sto studiando una poissioniana -> varianza = media -> errore
    statistico sul bin e' la deviazione standard -> errore stimato =
    radice(n)
164 d_sigma_residui = np.sqrt(d_oss)
165
166 #test del chi2
167 dof_3 = len(d_oss) - 1 #n di bin - 1 (vincolo sul numero totale di
    N)
168 chi2_3 = np.sum(d_residui ** 2 / d_aspettati)
169 p_value_3 = chi2.sf(chi2_3, dof_3)
170
171 #plot del terzo grafico + residui
172 ax3.plot(d_x_bin_centers, d_aspettati, "r-", color="red", label=rf'
    $d$ aspettati')
173 ax3.plot([], [], ' ', label=rf'$\chi^2$ / dof: {chi2_3:.2f} / {
    dof_3}')
174 ax3.plot([], [], ' ', label=rf'p-value: {p_value_3:.2f}')
175 ax3.legend()
176 ax3.set_ylabel('Frequenza')
177 ax4.axhline(0, color='black', linestyle='dashed')
178 ax4.errorbar(d_x_bin_centers, d_residui, yerr=d_sigma_residui, fmt=
    '.,')
179 ax4.grid()
180 ax4.set_ylabel('Residui (osservati - attesi)')
181 ax4.set_xlabel(rf'$d$')
182
183 #####studio della d(n)#####
184
185 dist_medie = []
186 error_medie = []
187 for i in val_n:

```

```

188     #crea un array momentaneo dove immagazzina le distanze per ogni
        ciclo di i
189     attuale_d = np.empty(run_per_n)
190
191     #eseguire il random walk con un numero di passi dato dall'indice
        i per run_per_n volte e salva la distanza ottenuta ad ogni run
192     for j in range(run_per_n):
193         x_i, y_i = random_walk(i)
194         attuale_d[j] = (np.sqrt(x_i**2 + y_i**2))
195
196     #calcola la media di delle distanze ottenute per ogni step e le
        salva nella lista dist_medie
197     media_d = np.mean(attuale_d)
198     dist_medie.append(media_d)
199
200     #calcola la deviazione standard della media e la salva nella
        lista error_medie
201     #ddof=1 per lo stimatore corretto della deviazione standard
        campionaria
202     std_d = np.std(attuale_d, ddof=1)
203     err_media = std_d / np.sqrt(run_per_n)
204     error_medie.append(err_media)
205
206     #converte le liste in array di numpy per semplicita'
207     dist_medie = np.array(dist_medie)
208     error_medie = np.array(error_medie)
209
210     fig4, (ax5, ax6) = plt.subplots(2, 1, sharex=True, height_ratios
        =[3, 1])
211
212     #calcolo dei residui
213     residui = dist_medie - (np.sqrt(np.pi)/2) * np.sqrt(val_n)
214
215     #normalizzo i residui per avere l'asse y dei residui in unita'
        dell'errore
216     residui_norm = residui/error_medie
217
218     #test del chi2
219     dof_4 = len(dist_medie)
220     chi2_4 = np.sum(residui_norm ** 2)
221     p_value_4 = chi2.sf(chi2_4, dof_4)
222
223
224     #plot del grafico
225     ax5.errorbar(val_n, dist_medie, yerr=error_medie, label=rf'$d(n)$',
        ,fmt='o')

```



```

226 ax5.plot(val_n, (np.sqrt(np.pi)/2) * np.sqrt(val_n), "r-", color='
      red', label=r'E[d]=$\frac{\{\sqrt{\pi}\}\{2\}\sqrt{n}\$')
227 ax5.plot([], [], ' ', label=rf'$\chi^2\$ / dof: {\chi2_4:.2f} / {
      dof_4}')
228 ax5.plot([], [], ' ', label=rf'p-value: {p_value_4:.2f}')
229 ax5.set_ylabel(rf'$d\$')
230 ax5.legend()
231
232 #plot dei residui
233 ax6.axhline(0, color='black', linestyle='dashed')
234 #se i residui sono normalizzati le barre di errore sono 1
235 ax6.errorbar(val_n, residui_norm, yerr=1, fmt='.')
236 ax6.grid()
237 ax6.set_xlabel(rf'$n\$')
238 ax6.set_ylabel(rf'Residui normalizzati [ $\sigma$ ]')
239
240 #blocco di salvataggio dei grafici
241 if img:
242     fig1.savefig('distribuzione_x.pgfig')
243     fig2.savefig('distribuzione_d2.pgfig')
244     fig3.savefig('distribuzione_d.pgfig')
245     fig4.savefig('distribuzione_d_n.pgfig')
246 else:
247     plt.show()

```

Listing 1: Codice random walk

4.1 Righe 1–64: Dichiarazioni

Nelle prime righe del codice vengono dichiarate:

- le librerie elencate nel Capitolo 3: ‘Materiale a disposizione’
- un piccolo blocco di codice utilizzato per stampare le immagini in formato *pgf*
- la funzione che codifica il random walk
- le funzioni modello delle distribuzioni: *Gaussiana*, *Esponenziale* e del χ
- le costanti usate nel codice

4.2 Righe 66–72: Generazione

In questo blocco di codice viene simulato un random walk per n passi N volte. Le coordinate (x_{finale}, y_{finale}) , per ogni run, vengono immagazzinate negli array x e y

4.3 Righe 74–103: Studio della x

Si sceglie di studiare la distribuzione della coordinata x , essa viene graficata mediante l'uso di un istogramma per identificarne la forma, il centro e dispersione dei dati rapidamente. La funzione *plt.hist* restituisce tre oggetti: *x_osservate* che contiene il numero di osservazioni per ciascun bin, *x.bins* dove vengono registrati gli intervalli dei bin e *'_'* che in gergo è detto *patches* dove sono salvate informazioni relative alla grafica. Vengono poi calcolati i centri dei bin: vengono presi tutti gli estremi dei bin tranne l'ultimo e sommati a tutti gli estremi dei bin meno il primo; la somma poi viene divisa per due. In questo modo si ottiene un array contenente le coordinate x dei centri di tutti i bin. Per ciascuna di queste coordinate viene quindi calcolato il valore atteso secondo una distribuzione gaussiana, con media 0 e varianza $\sigma^2 = \frac{n}{2}$. L'insieme di questi valori costituisce l'array *x_aspettate*.

Viene poi fatto un test del χ^2 per valutare la compatibilità dell'istogramma con la campana Gaussiana; per decretare i gradi di libertà viene preso il numero di osservazioni fatte e vi si sottrae 1 poichè: essendo il numero delle run (uguale, in somma, al numero delle osservazioni) una grandezza arbitraria e fissata, mi basta conoscere la somma delle altezze di tutti i bin meno l'ultimo per poter calcolare l'altezza dell'ultimo. Per completezza viene anche calcolato il p-value.

Infine si calcolano media e deviazione standard dell'array x . L'output grafico di questo studio è visibile in Figura 2

4.4 Righe 105–143: Studio della distanza al quadrato

Come prima cosa si calcola l'array delle distanze al quadrato per ogni run (dalla definizione di d : $d^2 = x_{finale}^2 + y_{finale}^2$). Come fatto nel punto precedente viene calcolato l'istogramma (e di conseguenza i valori osservati) e i valori aspettati mediante la funzione per una distribuzione esponenziale. A questo punto vengono calcolati i residui e la stima dell'incertezza. Per quest'ultima bisogna ragionare su quello che sta accadendo: per una singola run la probabilità che l'esito cada in un determinato bin è descritto da una binomiale. Non si sta però simulando una sola run, ma $N \gg 1$; questo ci porta a non valutare più la binomiale, bensì il suo limite matematico che converge perfettamente a una distribuzione Poissoniana. La proprietà fondamentale di tale distribuzione è che la sua varianza è identica al suo valore atteso ($\sigma^2 = \lambda$). L'incertezza su un conteggio è la radice quadrata della varianza, quindi risulta essere: $\forall k \in \text{residui} : \sigma_k = \sqrt{k}$

Anche in questo caso vengono calcolati χ^2 e p-value. Lo studio della distanza quadra è in Figura 3

4.5 Righe 145–181: Studio della distanza

Viene calcolata la distanza semplicemente prendendo la lista delle d^2 e facendo la radice quadrata di ogni elemento. Le considerazioni sono esattamente identiche a quelle fatte per le distanze al quadrato con l'unica differenza che le distanze, come visto nel Capitolo 2: ‘Cenni Teorici’, si distribuiscono come un χ . Lo studio delle distanze è rappresentato in Figura 4

4.6 Righe 183–238: Studio della distanza in funzione del numero di passi

Per studiare il variare della distanza in funzione del numero di passi si è deciso di eseguire l'algoritmo di random walk un numero fissato di volte variando in numero di passi. Il numero di passi varia tra *min_n* e *max_n* e per ogni incremento aumenta di *step_width*. Ad esempio se *min_n* = 10, *max_n* = 50 e *step_width* = 10 il valore del numero di passi seguirà questo andamento: 10 → 20 → 30 → 40 → 50. Per ogni valore di *val_n* (array che contiene i valori del numero di passi), verranno eseguiti *run_per_n* run di random walk con quel numero di passi. Per ogni step, inoltre, calcola la distanza media ottenuta dalle run e la salva in *dist_medie*, allo stesso modo calcola la deviazione standard per la media (che poi salva nell'array *error_medie*). Qui sorge un problema: non si sta calcolando per un intero insieme di dati ma si sta analizzando un campione per dedurre il comportamento generale. Utilizzo quindi la media campione, questo ci costringe ad utilizzare un grado di libertà in meno, in formula:

$$s = \sqrt{\frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N - 1}} \quad (6)$$

Dove: N è il numero di run per ogni passo di *val_n* (ovvero *run_per_n*), $\bar{x}$ è la media campione cioè *dist_medie* in quel passo e x_i sono tutti gli elementi di *attuale_d* ovvero le distanze trovate per quel passo. In termini di codice diventa: *np.std(attuale_d, ddof=1)*.

Per trovare l'incertezza di un singolo punto (ovvero di un singolo passo di *val_n*) devo dividere per il numero di campioni:

$$\sigma_{\bar{x}} = \frac{s}{\sqrt{N}} \quad (7)$$

In questo modo si riescono a trovare le medie per ogni valore di *val_n* e il relativo errore. I valori di aspettazione sono dati dall'Equazione 5, con questi è stato possibile calcolare i residui (che sono stati poi normalizzati) e fare il test del χ^2 e p-value. Lo studio è riportato in Figura 5

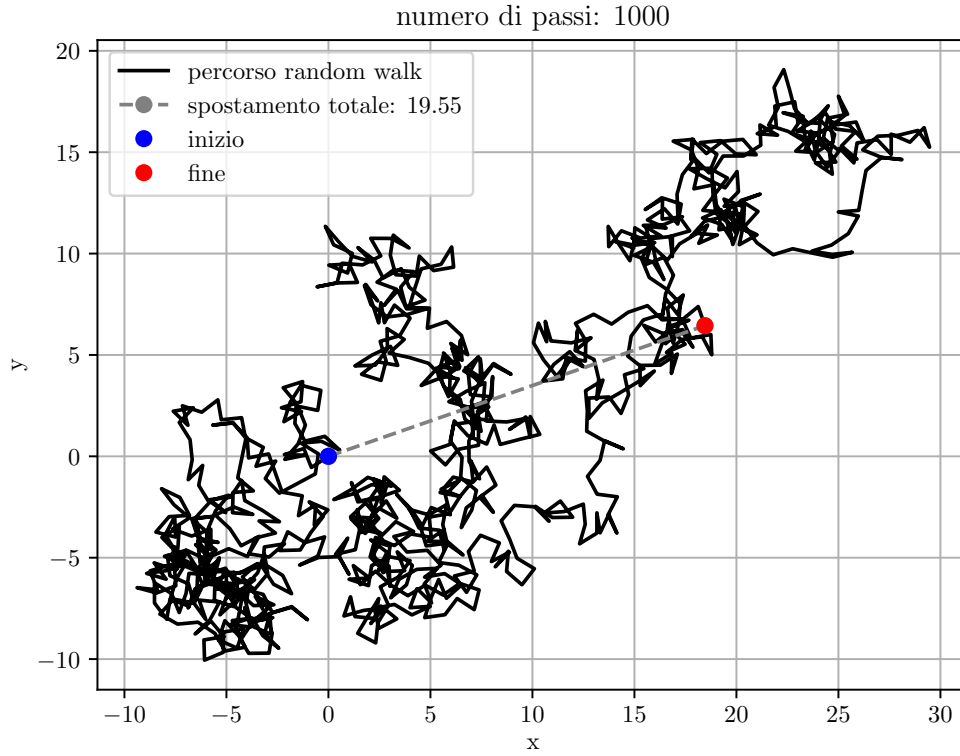


Figura 1: Pattern random walk

5 Analisi dati

5.1 Distribuzione della x

Osservando il grafico in Figura 2 appare chiaro che la coordinata x si distribuisca come una Gaussiana; ciò viene confermato dai test riportati in legenda.

Il $\chi^2_{atteso} = v \pm \sqrt{2v} = 78 \pm \sqrt{2 \cdot 78} \approx 78 \pm 12$ è compatibile con il $\chi^2_{osservato} = 82$.

Come secondo criterio di verifica dell'ipotesi si analizza la media: come riportato nel Capitolo 2: 'Cenni Teorici' ci si aspetta una media $\mu_{aspettata} = 0$.

La media osservata $\mu_{osservata} = -0.04$, per decretare la compatibilità si deve calcolare l'errore associato a questa misura. Si ricava lo stimatore della varianza dello stimatore della media per ottenere l'errore sul valor medio

$$\sigma_{\mu} = \sqrt{\frac{\sum_{i=1}^N (x_i - \mu_{osservata})^2}{N(N-1)}}$$

$$\text{Essendo } \mu_{osservata} = \frac{\sum_{i=1}^N x_i}{N}$$

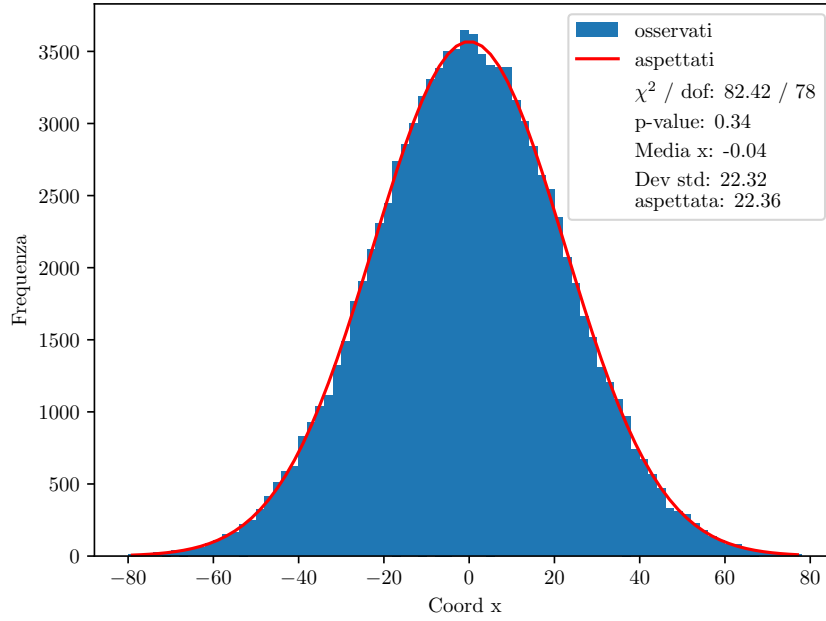


Figura 2: Distribuzione della coordinata x

$$\sigma_{\mu} = \frac{\sigma_{campionaria}}{\sqrt{N}} = \frac{22.32}{\sqrt{100000}} \approx 0.07$$

Quindi $\mu_{osservata} = -0.04 \pm 0.07$ è compatibile con $\mu_{atteso} = 0.00$

L'ultimo criterio di verifica è la deviazione standard della media: $s_{atteso} = 22.36$, se i dati di partenza sono distribuiti normalmente (ipotesi verificata dal Teorema del Limite Centrale), l'incertezza associata alla stima della deviazione standard è

$$\sigma_s = \frac{s_{atteso}}{\sqrt{2N}}$$

Sostituendo con i dati della simulazione: $\sigma_s = \frac{22.36}{\sqrt{200000}} \approx 0.05$

La deviazione standard osservata risulta essere $s_{osservata} = 22.32 \pm 0.05$ e quindi compatibile con il valore atteso. Le osservazioni fatte sono state schematizzate in Tabella 1

misura	valore atteso	valore misurato	compatibilità
χ^2	78 ± 12	82	compatibile
media (μ)	0.00	-0.04 ± 0.07	compatibile
deviazione std (s)	22.36	22.32 ± 0.05	compatibile

Tabella 1: Sintesi dello studio della coordinata x

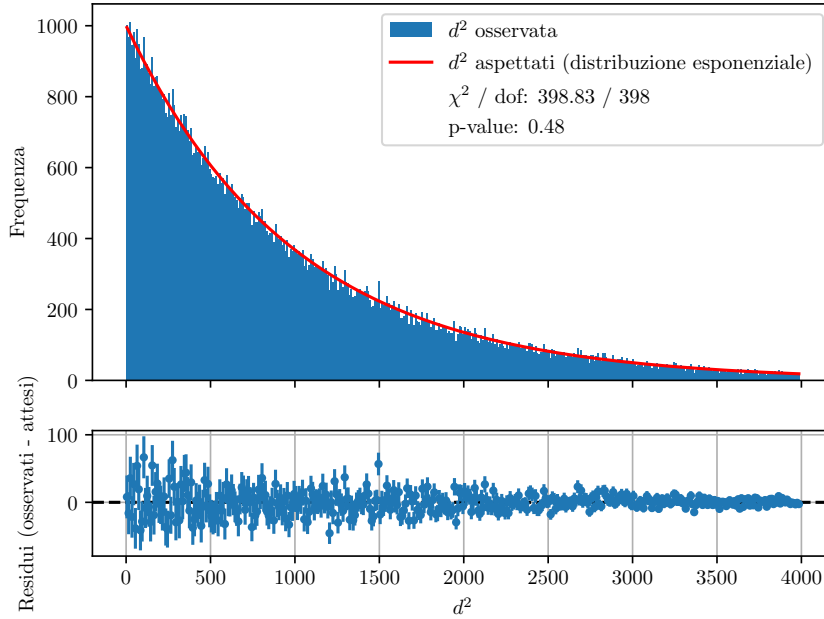


Figura 3: Distribuzione della distanza²

5.2 Distribuzione della distanza al quadrato

Dal grafico in Figura 3 si può notare come la d^2 si distribuisca come una distribuzione esponenziale. Per verificare la tesi si è eseguito un test del χ^2 che ha riportato i dati presenti in Tabella 2. Lo studio dei residui non evidenzia alcun comportamento anomalo.

misura	valore atteso	valore misurato	compatibilità
χ^2	398 ± 28	399	compatibile

Tabella 2: Sintesi dello studio della d^2

5.3 Distribuzione della distanza

Analizzando il grafico in Figura 4 risulta evidente che la distribuzione della distanza segue il modello della distribuzione del χ . È stato fatto un test del χ^2 in verifica della tesi, i risultati sono riportati in Tabella 3. Il test ha evidenziato come il χ^2 risulti troppo piccolo, è stato però calcolato anche il p-value, pari a 0.89, indicando così un'elevata compatibilità tra i dati ed il modello. Non vi sono evidenze per rifiutare il modello; tuttavia, il valore relativamente alto del p-value potrebbe suggerire una possibile sovrastima delle incertezze sperimentali.

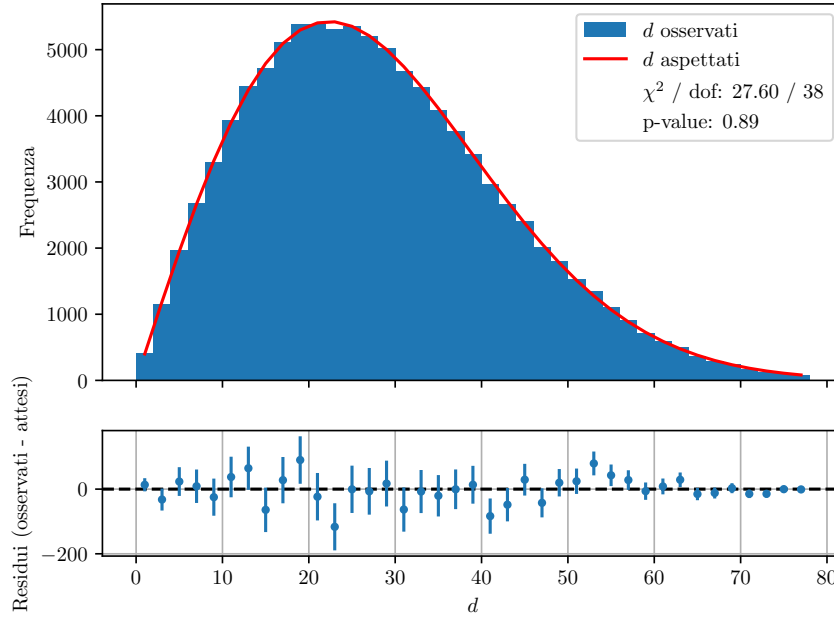


Figura 4: Distribuzione della distanza

misura	valore atteso	valore misurato	compatibilità
χ^2	38 ± 8.7	27.6	compatibile ²

Tabella 3: Sintesi dello studio della d

5.4 Distanza in funzione del numero di passi

Nel grafico in Figura 5 si nota come i punti seguono l'andamento ipotizzato. Anche in questo caso è stato realizzato un test del χ^2 , i risultati sono presenti in Tabella 4. Il grafico dei residui ci ha fatto ipotizzare che ci potesse essere un qualche errore sistematico, per confutare questa ipotesi sono state eseguite altre run di questo esperimento le quali hanno mostrato come i residui presenti in figura sono frutto del caso e non sono associabili ad errori sistematici.

misura	valore atteso	valore misurato	compatibilità
χ^2	50 ± 10	44	compatibile

Tabella 4: Sintesi dello studio della distanza in funzione del numero di passi

²Anche se il χ^2 è piccolo non si può rigettare il modello poichè la p-value sottolinea una compatibilità seppur con un errore che potrebbe essere sovrastimato

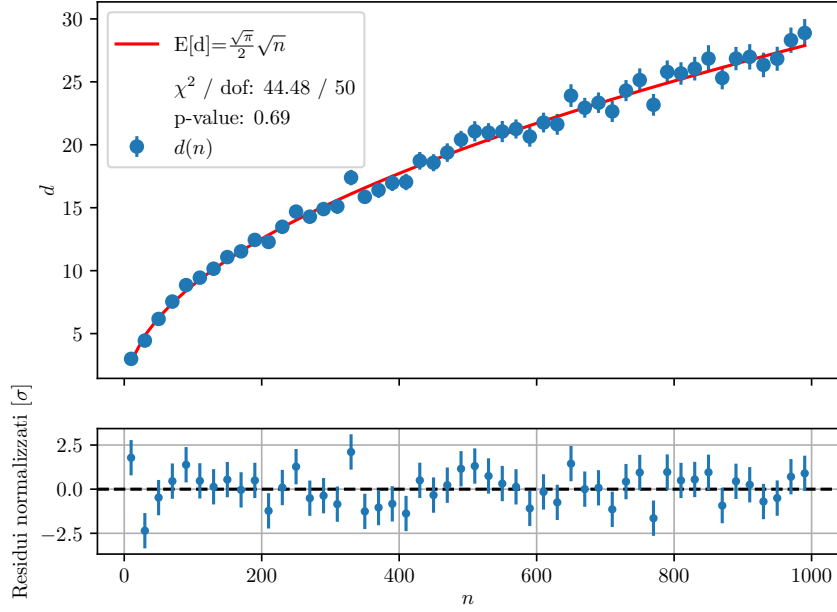


Figura 5: Grafico della distanza in funzione del numero di passi

6 Conclusioni

Dalle analisi fatte si evince che i modelli ipotizzati per le distribuzioni della coordinata x , della distanza quadra e della distanza, per un algoritmo di Random Walk in 2 dimensioni, sono corretti, dimostrando che l'algoritmo rappresenta effettivamente un moto browniano. Si è anche dimostrato sperimentalmente come la distanza raggiunta da n passi di random walk è in media $\sqrt{n}$ a meno di una costante ($\frac{\sqrt{\pi}}{2}$). Per quanto riguarda la distribuzione della distanza, l'errore potrebbe essere sovrastimato, potremmo provare a diminuire la larghezza dei bin dell'istogramma in Figura 4.